TLB and Page Table Simulator: Technical Documentation

This document describes how the simulator is built and how its parts work. For setup and usage, see the main `README.md`. For the full write-up, see the project report.

1. Purpose

The simulator demonstrates virtual-to-physical address translation using a page table and a TLB (Translation Lookaside Buffer). It prints every step of each translation so the process can be followed and explained.

2. Configuration constants

Defined at the top of `tlb_simulator.c`:

Constant	Value	Meaning
`OFFSET_BITS`	12	Bits used for the offset inside a page
`PAGE_SIZE`	4096	Page and frame size in bytes (1 << 12)
`NUM_PAGES`	16	Number of virtual pages
`NUM_FRAMES`	8	Number of physical frames
`MAX_TLB`	8	Largest allowed TLB size

The TLB size (1 to 8) and the policy (LRU or FIFO) are chosen when the program starts.

A 16-bit virtual address is laid out as:

```
 15        12 11                0
+------------+-------------------+
| page (4 b) |   offset (12 b)   |
+------------+-------------------+
```

Physical address = (frame number << 12) | offset.

3. Data structures

PageTableEntry (array of 16, one per virtual page)

Field	Meaning
`valid`	1 if the page is in physical memory
`frame`	Frame number holding the page
`loaded`	Clock value when the page was loaded (used by FIFO)
`last_used`	Clock value of the most recent use (used by LRU)

TLBEntry (array of up to 8)

Field	Meaning
`valid`	1 if the slot is in use
`page`, `frame`	The cached translation
`loaded`, `last_used`	Timestamps for FIFO and LRU

frame_owner[8] records which page occupies each frame, or -1 if the frame is free.

clock_tick is a global counter that increases by one on every access. It provides the timestamps.

4. Translation flow

```
Virtual address
      |
 split into page + offset
      |
 search TLB ----found----> TLB HIT -> frame
      |
   not found
      |
 check page table --valid--> TLB MISS -> frame
      |
   invalid
      |
 PAGE FAULT: take a free frame, or evict a victim page
      |
 add the translation to the TLB (evict a TLB entry if full)
      |
 physical address = (frame << 12) | offset
```

5. Replacement policies

The victim is the entry with the **smallest key**:

Policy	Key	Behaviour
LRU (Least Recently Used)	`last_used`	Every access refreshes the timestamp
FIFO (First In, First Out)	`loaded`	Timestamp is set once, when loaded

The same rule is used for choosing a TLB victim and a page victim. When a page is evicted from memory, its TLB entry is also removed, so the TLB never holds a stale translation.

6. Functions

Function	Description
`reset()`	Clears the page table, TLB, frames and statistics
`key_tlb()`, `key_pte()`	Return the replacement key for the current policy
`tlb_insert()`	Adds a translation, evicting a victim if the TLB is full
`tlb_flush_page()`	Removes a page from the TLB
`handle_page_fault()`	Finds a free frame or evicts a victim, then loads the page
`translate()`	Full translation; returns `TLB_HIT`, `TLB_MISS` or `PAGE_FAULT`
`show_tables()`	Prints TLB, page table and frame contents
`show_stats()`	Prints access counts and the TLB hit ratio
`random_address()`	Random address; about 70% from pages 1, 2, 5, 9 and 10 to imitate locality
`configure()`	Asks for TLB size and policy
`run_file()`	Translates every hex address in a text file
`main()`	Interactive loop or file mode, depending on arguments

7. Input and output

Input: hex addresses from 0x0000 to 0xFFFF, either typed or read from a file (one per line; blank or invalid lines are skipped, and out-of-range values are reported and skipped).

Output for each translation: page number and offset, whether the TLB hit, any page table lookup, page fault, eviction or TLB update, and the final physical address with the result type.

Statistics: total accesses, TLB hits, TLB misses, page faults and TLB hit ratio, where hit ratio = hits / total accesses.

A TLB miss means the page was in memory but not in the TLB. A page fault means the page was not in memory.

8. Testing summary

Scenario	Settings	Result
First access, repeat, same page different offset	TLB 4, LRU	Fault, hit, hit
Sample file (8 addresses)	TLB 4, LRU	3 hits, 0 misses, 5 faults, 37.5%
TLB miss with valid page (pages 1, 2, 1)	TLB 1, LRU	Fault, fault, miss
Pages 1, 2, 1, 3, 1	TLB 2, LRU	2 hits, 0 misses, 3 faults
Pages 1, 2, 1, 3, 1	TLB 2, FIFO	1 hit, 1 miss, 3 faults
Pages 0 to 7, then 8, then 0	TLB 4, LRU	Page 0 evicted, then page 1 evicted
Invalid input (0x10000, text)	Any	Error shown, statistics unchanged

9. Limitations

- One process and a single-level page table.
- Sizes are compile-time constants (except TLB size).
- No dirty or reference bits, and no data is actually stored in the simulated frames.

10. Possible extensions

- Multi-level page tables.
- Optimal replacement policy for comparison.
- Multiple processes, each with its own page table.
- Command-line options for sizes and policy.
- A graphical interface.

11. Safety

The simulator is a normal user-space program. It only modifies its own arrays and does not touch real system memory or other processes.